

THÈSE

Pour obtenir le grade de

DOCTEUR DE L'UNIVERSITÉ GRENOBLE ALPES



École doctorale : EEATS - Electronique, Electrotechnique, Automatique, Traitement du Signal

Spécialité : Nano électronique et Nano technologies

Unité de recherche : CEA /LIST - Laboratoire d'intégration de systèmes et de technologies

Conception d'un Système Mémoire pour Traitement de Données Creuses

Design of a Memory System for Sparse Data Processing

Présentée par :

Valentin ISAAC--CHASSANDE

Direction de thèse :

Frédéric ROUSSEAU

PROFESSEUR DES UNIVERSITES, Grenoble INP - UGA

Adrian EVANS

CEA

Directeur de thèse

Co-encadrant de thèse

Rapporteurs :

Thèse soutenue publiquement le , devant le jury composé de :



Valentin ISAAC--CHASSANDE

Université Grenoble Alpes
CEA Grenoble (DRT/LIST/DSCIN/LSTA)
TiMA Laboratory (SLS)

ORCID : 0009-0007-9842-5777

valentin.isaacchassande@univ-grenoble-alpes.fr

valentin.isaacchassande@cea.fr

THESIS

DESIGN OF A MEMORY SUB-SYSTEM FOR SPARSE DATA PROCESSING

September 30, 2025

Thesis Adviser: Pr. Frédéric ROUSSEAU
Thesis Co-Supervisors: Dr. Adrian EVANS
Dr. Yves DURAND

CSI Members: Pr. Yves ROBERT (HDR)
Pr. Christophe PICARD

EEATS Doctoral College, NENT Speciality
2022 – 2025 Academic Years
From 10.25.2022 to 10.24.2025

Acknowledgements

TODO

“Work. Or school. This is for the money. But what is anything else, for that matter, but required means of enhancing your bare living – better sleep and better food – like spending decades just to turn a little knob higher on a pathetic radio called existence.”

Karl Kristian Flores, The Goodbye Song.

Résumé

TODO

Abstract

TODO

List of Publications

This thesis is based on the following publications:

- I Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. Reference [23]
- II Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–7. Reference [24]
- III **TODO**

The papers will be referred to in the thesis using their Roman numerals.

Summary

Acknowledgements	1
Epigraph	2
Résumé	3
Abstract	4
List of Publications	5
Summary	6
Notations	8
Glossary	9
Acronyms	11
Introduction	12
1 Background	14
1.1 Introduction	14
1.2 Sparse Matrices	15
1.3 Sparse Formats	16
1.4 Algorithms for Dense and Sparse Matrix Multiplication	17
1.5 Hardware Challenges for Sparse Matrix Computation	21
1.6 Conclusion	27
Conclusion	28
1 TODO	28
Appendices	29
A Sparse Formats	30
B A Generic Representation for <i>Sparse Formats</i>	31
C Extended Overview of the State-of-the-Art Accelerators for Sparse Computing . .	32
D Analytical Model of a <i>Reduction Cache</i> Computing an OP SpMV Kernel	33
E Python Model of a Data-Cache with Support for <i>Reductions</i>	34
F Statistics	35
List of Figures	36
List of Tables	37
List of Algorithms	38
List of Source Code	39
Bibliography	40

Contents**44**

Notations

$$\forall (a, b) \in \mathbb{Z}^2, \llbracket a, b \rrbracket = [a, b] \cap \mathbb{Z}$$

nnz , M , A , b and c , $\overline{nnz_r}$, d_r ...

TODO

Glossary

- band** Refers to an area of a matrix that is concentrated around the diagonal. 9, 15, 16, 17
- banded** Refers to a matrix structure where the non-zero elements are concentrated around the diagonal. 15, 16, 27, 36
- bandwidth** With this typography, refers to the size of the band of a matrix. 15
- common rank** Refers to a rank that is common between two data-entries, when computing a kernel. 17, 18, 21, 24
- dense format** Is the uncompressed format where all zeros and non-zeros are stored, without any use of metadata. 16, 22, 23
- fiber** Is the generic term to describe a one-dimensional stream of data or metadata. It corresponds to a row or column in the case of a two-dimensional matrix. A stream of indices is a *fiber* of metadata. 21, 24
- gather** Refers to reading data from memory. We specifically use this term to categorize data transfers that are irregular. 22, 24, 27
- ineffectual operation** Is an operation that leads to a neutral result, 0 for an addition or 1 for a multiplication. 21
- intersection** Refers to the process of finding non-zero partial sums, when computing a matrix multiplication. 18, 19, 20, 21, 22, 24, 26, 27, 38
- irregular access** See “random” access. 9, 14, 27
- irregular data** Refers to data that has low spatial and temporal locality, thus being more likely to generate irregular accesses. 15, 27
- metadata** Is data that provides information about other data, such as index arrays of sparse formats. 9, 10, 16, 17, 21, 24, 31
- partial sum** Refers to the intermediary result before updating the output, when computing a matrix multiplication. 9, 18, 21, 22, 24
- ‘perfectly’ banded** Refers to a matrix structure where all the non-zero elements are concentrated within a delimited area around the diagonal (the band). 15
- “random” access** Is a memory access that occurs on an arbitrary memory address (not necessarily consecutive in memory with previous and next accesses). 9, 14
- “random” update** Is a “random” access where the access is a read, modify and write on a given address. It is similar to a “random” *reduction*. 21
- rank** Refers to a dimension of the kernel. 9, 16, 17, 18, 20, 24
- reduction** Refers to the process of updating an entry. 9, 18, 21, 22, 23, 24, 26, 27, 38

scatter Refers to updating data in memory. We specifically use this term to categorize data transfers that are irregular. 24, 27

sequential accesses Are a group of memory accesses (from a data array, for example) that occur on consecutive memory addresses, in order through time. 16

sparse format Is a compression format for sparse matrices, which avoids storing zeros by using metadata. 9, 16, 17, 19, 20, 21, 27, 31

tile Refers to a sub-region of a *tiling* process. 20, 26

tiling Is a method consisting in splitting data into several independent sub-regions to be processed separately. 10, 20, 22, 24, 26, 27, 31, 36, 38

Acronyms

BaCSC *Banded Compressed Sparse Column.* 16, 17, 27, 36

BCSR *Blocked Compressed Sparse Row.* 16

COO *COOrdinate.* 16, 17, 23, 27, 31, 36

CSC *Compressed Sparse Column.* 16, 17, 21, 22, 23, 24, 27, 31, 36, 38

CSR *Compressed Sparse Row.* 16, 17, 20, 21, 22, 23, 24, 27, 31, 36, 38

DIA *DIAGonal.* 16

ELL *ELLPACK.* 16

Gust *Gustavson.* 17, 18, 19, 20, 22, 24, 25, 26, 27, 37, 38

HYB *HYBbrid.* 16

IP *Inner-Product.* 17, 18, 19, 20, 21, 22, 24, 25, 26, 27, 37, 38

MM *Matrix-Matrix.* 17, 18, 19, 24, 27, 38

MV *Matrix-Vector.* 17, 18, 19, 20, 24, 27, 38

OP *Outer-Product.* 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 37, 38

PO *partial output.* 18, 19, 20, 21, 22, 24, 27

SpGEMM *General Sparse Matrix-Matrix multiplication.* 17

SpGEMV *General Sparse Matrix-Vector multiplication.* 17

SpMM *Sparse Matrix-Matrix multiplication.* 17

SpMSpM *Sparse Matrix-Sparse Matrix multiplication.* 17, 19, 21, 22, 23, 24, 25, 26, 27, 37, 38

SpMSpV *Sparse Matrix-Sparse Vector multiplication.* 17

SpMV *Sparse Matrix-Vector multiplication.* 17, 20, 27, 36

Introduction

THE rapid growth of data-intensive applications has made sparse and irregular data structures increasingly prevalent in scientific computing, machine learning, and graph analytics. However, efficiently processing sparse matrices remains a fundamental challenge due to their inherent irregularity, which leads to unpredictable memory access patterns, limited opportunities for data reuse, and suboptimal utilization of hardware resources. Existing hardware accelerators have demonstrated partial solutions, yet they often rely on restrictive assumptions, exhibit limited scalability, or fail to fully exploit the memory bandwidth.

This thesis addresses the following central question:

PROBLEM STATEMENT

How can hardware and architectural mechanisms be designed to efficiently support the computation of irregular sparse data, in particular random reductions, while ensuring scalability, adaptability, and performance across diverse workloads?

To tackle this problem, we investigate the limitations of existing algorithms and accelerators, propose a novel cache architecture with support for reduction operations, tailored to sparse data. We then evaluate the behaviour of this specialized cache through modeling, simulation, and hardware prototyping.

This manuscript is structured around several key contributions. We begin with an in-depth discussion of irregular data, with a particular emphasis on sparse matrices (Chapter 1). This includes a study of fundamental sparse matrix multiplication kernels and their associated algorithms, highlighting the main hardware challenges that arise when dealing with sparse data. A high-level analysis of algorithms for sparse data computing is provided, with a focus on basic storage schemes and opportunities for data reuse. Building on this foundation, we review the state-of-the-art in hardware accelerators designed for efficient sparse matrix computing (Chapter ??). This review identifies common characteristics, proposes a classification of existing accelerators and design tendencies, offers a comparative analysis, and identifies the major tendencies across designs. This analysis of the state of the art guided our subsequent architectural choices.

Following this survey, we introduce a novel cache architecture specifically designed to address the challenge of “random” reductions (Chapter ??). The proposed cache integrates reduction support and organizes its memory into multiple regions optimized for denser and sparser data regions. A Python model was developed to simulate memory transactions in this cache and to compare its behavior with that of a generic cache. To further investigate its performance, we complement this analysis with a probabilistic model, implemented in C, that enables rapid simulation and design space exploration (Chapter ??). At the micro-architectural level, we detail the virtual partitioning of cache memory into multiple configurable regions, which ensures both flexibility and adaptability to diverse workloads (Chapter ??).

The proposed cache is then evaluated through RTL simulation, which reveals two distinct classes of matrices that exhibit different performance profiles (Chapter ??). Finally, the manuscript outlines potential future enhancements to improve both performance and scalability, including a multicore extension of the cache architecture and optimizations targeting bandwidth efficiency (Chapter ??).

Contributions of the Thesis

The main contributions of this thesis can be summarized as follows:

1. **Analysis of irregular and sparse data computing** (Chapter 1)
 - Study of fundamental sparse matrix multiplication kernels and algorithms.
 - Identification of the key hardware challenges associated with sparse data processing.
 - High-level analysis of algorithms for sparse data computing, with emphasis on storage schemes and data reuse opportunities.
2. **Survey and classification of state-of-the-art accelerators** (Chapter ??)
 - Comprehensive review of hardware accelerators for sparse matrix computing.
 - Identification of common architectural features and design tendencies.
 - Comparative analysis to explain design choices under varying objectives and constraints.
3. **Design of a cache architecture for random reductions** (Chapter ??)
 - Proposal of a cache with native reduction support, capable of handling both dense and sparse data via multiple cache regions.
 - Development of a Python-based simulation model to analyze memory transactions and benchmark against a generic cache.
4. **Probabilistic modeling and exploration of cache behavior** (Chapter ??)
 - Implementation of a lightweight C-based probabilistic model for rapid simulation.
 - Exploration of cache dimensioning and behavior across diverse workloads.
5. **Micro-architecture of the reduction cache** (Chapter ??)
 - Introduction of a virtual memory partitioning scheme enabling multiple configurable regions.
 - Emphasis on flexibility and adaptability in cache design.
6. **Evaluation through RTL simulation** (Chapter ??)
 - Assessment of the proposed cache architecture using RTL-level simulation.
 - Identification of two distinct groups of sparse matrices leading to different performance profiles.
7. **Future directions for performance and scalability** (Chapter ??)
 - Proposal of a multicore version of the cache to extend scalability.
 - Exploration of bandwidth optimization strategies.

Chapter 1

Background

Contents

1.1	Introduction	14
1.2	Sparse Matrices	15
1.2.1	Banded Matrices	15
1.3	Sparse Formats	16
1.4	Algorithms for Dense and Sparse Matrix Multiplication	17
1.4.1	Tiling	20
1.5	Hardware Challenges for Sparse Matrix Computation	21
1.5.1	Inner-Product Algorithm	21
1.5.2	Outer-Product Algorithm	21
1.5.3	Summary	22
1.5.4	SpMSpM Algorithm Analysis	24
1.6	Conclusion	27

1.1 Introduction

MODERN computing systems are designed to deliver high performance by exploiting memory hierarchies, parallelism, and increasingly sophisticated hardware features. However, the efficiency of these systems is strongly tied to the regularity of memory access patterns. Applications with predictable, sequential data access can fully benefit from cache locality, prefetching mechanisms, and vectorization. In contrast, applications characterized by irregular accesses, where memory locations are not known in advance or follow non-contiguous patterns, are not able to achieve peak performance.

In domains such as graph analytics, sparse linear algebra, unstructured mesh computations, and database queries, the memory access patterns are often irregular. In these applications, pointer chasing, indirect indexing, and dynamically changing data structures disrupt spatial and temporal locality. As a result, the hardware struggles to hide the latency of memory accesses, leading to poor cache utilization and underutilization of compute resources.

The impact of irregular accesses extends beyond raw performance. They also complicate algorithm design, hinder scalability on many-core architectures, and increase energy consumption due to repeated off-chip memory requests. Understanding and addressing these challenges is therefore a central concern in both computer design and application optimization. This chapter provides the necessary background for analyzing irregular accesses, highlighting their sources, effects, and the architectural considerations they expose, focusing on the case of indirect indexing in sparse matrix computation. The content of this chapter is partially based on the content from Papers I and II.

DEFINITION

Irregular accesses, or “**random**” **accesses**, are sequences of accesses where the sequence can not be easily predicted.

1.2 Sparse Matrices

IN most applications, irregular data can be found in matrix computing, where large matrices filled with many zeros are stored in compressed format in memory. These sparse matrices are commonly used in diverse fields such as linear algebra, where sparse matrix multiplication is used to solve linear systems of equations for scientific purposes or simulation, as well as graph analytics where the vertices of a graph are represented as a sparse matrix [8], or finally transformers or other similar AI features where weights and activations can be represented and processed as sparse matrices [15, 16]. In these contexts, matrices are large, with dimensions up to billions of elements, and highly sparse, with non-zero densities down to 10^{-7} [30].

A matrix is defined as a two dimensional array. We note matrices with capital letters A, B, C , conversely to vectors a, b, c . Their dimension are denoted using M, N and K . For example, we can define a square matrix A of dimension M , or a matrix B of dimension (K, N) . We note nnz the number of non-zero elements of a matrix, and d the associated density. With the example of a matrix $A(M, N)$:

$$d_A = \frac{nnz_A}{M \times N}$$

DEFINITION

A **sparse matrix** is a matrix filled with many zeros.

NOTE

In general, we consider a matrix to be highly sparse when its number of non-zero elements has the same order of magnitude than its dimension, $nnz \sim M$. But there is no intrinsic definition of the frontier between a sparse and a dense matrix. In this manuscript, we consider a matrix as sparse when it needs to be compressed in memory (see Section 1.3).

1.2.1 Banded Matrices

Matrices used in scientific computation originate from physical modelling where a system of partial differential equations is discretized, resulting in a large sparse matrix. In these systems, the forces between elements decreases quickly with their distance, creating banded matrices. Moreover, numerical techniques exist to transform matrices into banded matrices [33, 34, 44].

A banded matrix is a matrix which has a dense region around its main diagonal and which is sparser away from the diagonal. To give a few visual examples, Figure 1.1 shows the structure plots for four typical matrices found in the SuiteSparse Matrix Collection [30], having a high-density band along the diagonal. In Figure 1.2, we illustrate a banded, square matrix of dimension M . We define w_B as the width of the banded region, or *bandwidth*, as shown in red in the figure. Sometimes it is easier to refer to the *half-bandwidth*, w_{hB} , which respects the relation $w_B = 2 \times w_{hB} - 1$. The average density in the band and out of the band is defined as d_{IB} and d_{OB} , respectively. As in Figure 1.1d, in a ‘perfectly’ banded matrix, there are no elements outside the band, thus $d_{OB} = 0$.

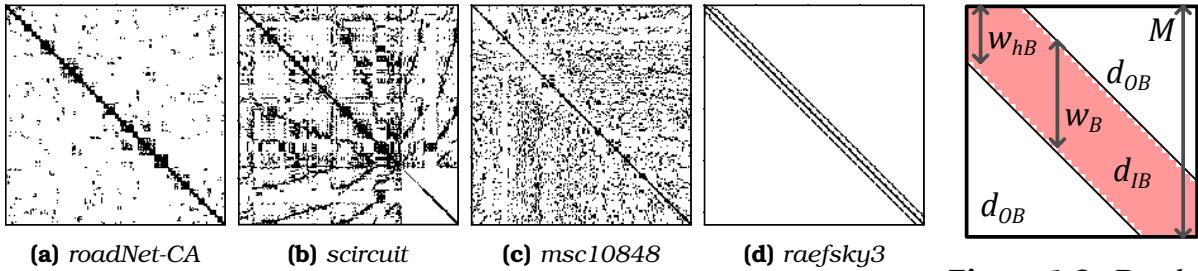


Figure 1.1: Examples of Banded Matrix Structure

Figure 1.2: Banded Matrix Definition

DEFINITION

A **banded matrix** is a sparse matrix which has a higher density of non-zero elements around the diagonal. A ‘**perfectly**’ **banded matrix** concentrates all its non-zero elements within a

delimited region around the diagonal. This region is called the band.

NOTE

The SuiteSparse Matrix Collection [30] is a large, open source data base of sparse matrices from multiple real-world applications, many of which were provided by companies such as Boeing. It gathers around 3,000 matrices of various sizes, densities and structure, which are extensively used for benchmarking (see Chapter ??).

1.3 Sparse Formats

IN the absence of a compressed format, matrices are stored in fixed-size two-dimensional arrays. With this dense format, elements can be randomly accessed ($\mathcal{O}(1)$), and sequential accesses are highly efficient. The two dimensions are called ranks which correspond to the rows and columns. For matrices, there are thus two variants of the dense format, depending on which rank is stored sequentially in memory. In a *row-wise* (or *row-major*) storage, the elements within the rows are sequential, and conversely for *column-wise* (or *column-major*).

Sparse matrices contain a very large number of zeros, and to avoid storing these repeated zeros, there exist many compressed formats [4, 5, 11, 27, 39, 41], where certain formats are best-suited for matrices with a specific structure. The main principle of these sparse formats is to avoid storing the zero elements, but this requires additional metadata which gives the position of the non-zero elements.

For example, let us consider the commonly used *COOrdinate* (COO) format that is relatively simple. It consists of three tables of size nnz with only the non-zero elements (*val* table in Figure 1.3a) and their row and column indices (*row* and *col idx* tables). The tuples do not necessarily need to be sorted, which makes it easy to append new entries. However, if the tuples are not sorted, this format is not well suited for sequential traversal or locating a specific matrix entry. This format can be sorted *row-wise* as in the example in Figure 1.3a or *column-wise* if needed, but in this case, it is less efficient than the two other widely used sparse formats *Compressed Sparse Row* (CSR) (Figure 1.3b) and its *column-major* counterpart *Compressed Sparse Column* (CSC) (Figure 1.3c), which are sorted by construction. With CSC, the *val* table of size nnz stores the non-zero elements sorted *column-wise* and the *idx* table of size nnz stores the corresponding row indices, similarly to COO. The third table *ptr* of size $M + 1$ stores the positions of the columns. For example, in Figure 1.3c, the second column of index 1 (green) has non-zero elements between positions 2 and 4 (blue, 4 excluded) of tables *idx* and *val*. This *ptr* table allows CSR and CSC to have a lower memory footprint than COO.

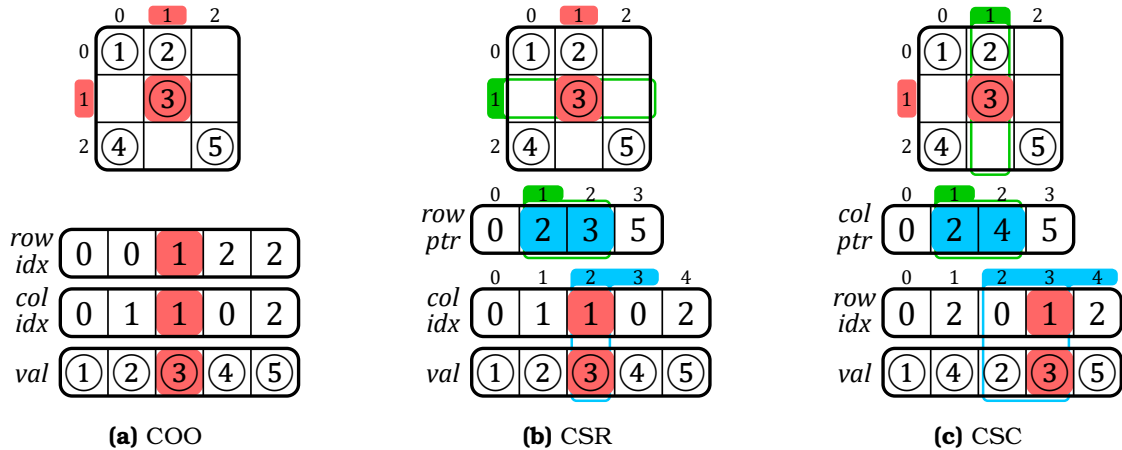


Figure 1.3: Example of a Sparse Matrix in COO, CSR and CSC Formats

Many others classes of sparse formats exists such as *Blocked Compressed Sparse Row* (BCSR), *DIAGonal* (DIA), *ELLPACK* (ELL) and *HYBbrid* (HYB), which all have their own advantages depending on the matrix structure or the hardware setup. Additional information about these other formats is provided in Appendix A.

In general, there are many derivatives from the above main categories of sparse formats. Ultimately, we can create any format we want depending on the software and hardware constraints. Later in this manuscript, we propose our own custom sparse format called *Banded Compressed Sparse Column* (BaCSC) that is an extension of CSC for banded matrices. As

shown in Figure 1.4, we added extra information in the *ptr* table to identify the beginning and end of the band. Indeed, three pointers instead of one are now needed per column, as shown with the example of column 2 in *yellow*. The column is split in one in-band (IB) region in the middle and two out-of-band (OB) regions around. The *ptr* table size thus becomes $3 \times M + 1$. With this format, we can avoid the computational cost of identifying in which region non-zero elements are, while traversing the matrix, a feature we take advantage of in Chapter ??.

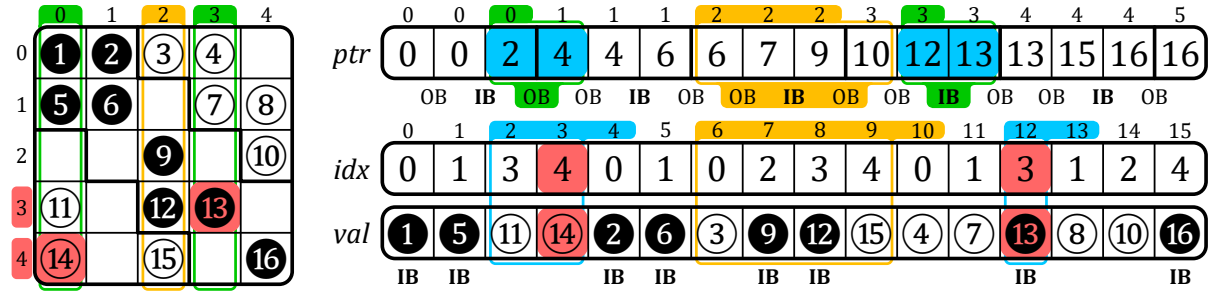


Figure 1.4: Example of a Sparse Matrix in BaCSC Format

DEFINITION

A **sparse format** is a memory representation of a sparse matrix which avoids storing zeros by using metadata.

NOTE

The most commonly used sparse formats are COO, CSR and CSC.

CONCLUSION

With the use of sparse formats, it is possible to greatly **decrease the memory footprint** of a sparse matrix while increasing the data dimensions. These formats **can be tailored** based on the needs of the software and hardware.

1.4 Algorithms for Dense and Sparse Matrix Multiplication

THE main matrix multiplication kernels that are of interest are Matrix-Vector (MV) and Matrix-Matrix (MM), which include SpMV and SpMSPV ($A \times b = c$), SpGEMV ($A \times b + d = c$), SpMM and SpMSPM ($A \times B = C$), and SpGEMM ($A \times B + D = C$), with ‘Sp’ for *sparse* and ‘GE’ for *general*. SpMV, as well as its variant SpGEMV, are by far the dominant operations in modern algebraic kernels (linear solvers, eigensolvers) which have been explicitly based on them [13, 18, 21, 25, 29, 36, 48]. Since these kernels are ubiquitous, their efficient implementation is primordial. In contrast, the SpMSPM multiplication is less frequent in standard algebraic kernels, but it is extensively used in algebraic graph manipulation [2, 7, 10, 12, 17, 26, 37], as well as in multigrid solvers [1, 6, 32, 38]. The growing importance of large graphs has motivated new research on optimizing SpMSPM applications.

In the following sections, we focus on SpMV and SpMSPM, with the notation $A(M, K) \times b(K) = c(M)$ and $A(M, K) \times B(K, N) = C(M, N)$, respectively. The rank K is the common rank between the two inputs A and B (or b). The existence of different ranks allows several algorithmic possibilities to compute matrix multiplication: the *Inner-Product (IP)* algorithm, the *Outer-Product (OP)* algorithm and *Gustavson’s (Gust)* algorithm [14, 20]. In all cases, the matrix A is traversed sequentially. The three algorithms, however, impose different access patterns for B (or b) and C (or c). Indeed, in many cases, they have to be accessed multiple times, as discussed in the following sections.

NOTE

In this manuscript, we focus on only two kernels, SpMV and SpMSPM. Note that, among MV kernels, SpMV is the most used and we specifically work on this kernel in Chapter ??. SpMSPM is also widely used and studying this MM kernel is of interest as the memory access patterns in both matrices are irregular.

The *general* (‘GE’) version of these kernels is not fundamentally different, as the majority of

the compute operations are in the matrix multiplication, so going forward, we will ignore the ‘GE’ variants.

1.4.0.1 Inner-Product Algorithm

The *Inner-Product* algorithm [14] is an intuitive way to calculate a matrix multiplication because it follows the loop order of the mathematical formulas (1.1) and (1.2) for MV and MM, respectively. The common rank K is the innermost loop of the algorithm (see Algorithms 1.1 and 1.2, lines 2 and 3, respectively). The output C (or c) is always updated sequentially, but the B -matrix (or b -vector) will be read entirely M times. In the context of sparse matrices, the reads of B (or b) are conditioned by the non-zero elements of A , which requires indirect accesses. If B (or b) is also sparse, the algorithm needs to perform *intersections* between the non-zero elements in the rows of A and the columns of B (or in the b -vector) (see Section 1.5). In Algorithm 1.2, exchanging M and N (lines 1 and 2) simply causes the output to be updated *column-wise*.

$$\forall i \in \llbracket 0, M-1 \rrbracket, c_i = \sum_{k=0}^{K-1} A_{i,k} \times b_k \quad (1.1)$$

$$\forall (i, j) \in \llbracket 0, M-1 \rrbracket \times \llbracket 0, N-1 \rrbracket, C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} \times B_{k,j} \quad (1.2)$$

Algorithm 1.1: Dense Matrix-Vector
Inner-Product Algorithm

```

1 for  $i \in \llbracket 0, M-1 \rrbracket$  do
2   for  $k \in \llbracket 0, K-1 \rrbracket$  do
3      $c_i += A_{i,k} \times b_k$ 
```

Algorithm 1.2: Dense Matrix-Matrix
Inner-Product Algorithm

```

1 for  $i \in \llbracket 0, M-1 \rrbracket$  do
2   for  $j \in \llbracket 0, N-1 \rrbracket$  do
3     for  $k \in \llbracket 0, K-1 \rrbracket$  do
4        $C_{i,j} += A_{i,k} \times B_{k,j}$ 
```

1.4.0.2 Outer-Product Algorithm

The *Outer-Product* algorithm [14] has the opposite rank loop order than the *Inner-Product*. Indeed, Algorithms 1.3 and 1.4 show that the common rank K is the outermost loop (line 1). It changes nothing mathematically except that the B -matrix (or b -vector) is always read sequentially. In the context of sparse matrices, the output C (or c) is updated with indirect accesses. The algorithm needs to perform *reductions* of the partial sums $A_{j,k} \times B_{k,j}$ (or $A_{j,k} \times b_k$), as we show in Section 1.5. To avoid irregularity when updating C (or c), the **OP** algorithm is often separated into two phases. First, during the *multiply* phase, we compute several *partial outputs* (POs) consisting of intermediate matrices (or vectors) of partial sums $A_{j,k} \times B_{k,j}$ (or $A_{j,k} \times b_k$), then during the *merge* phase, we accumulate all the POs to form the final C -matrix (or c -vector). With this method, **OP** necessitates the generation of at most K POs of densities conditioned by the non-zeros elements of both inputs A and B (or b).

Algorithm 1.3: Dense Matrix-Vector
Outer-Product Algorithm

```

1 for  $k \in \llbracket 0, K-1 \rrbracket$  do
2   for  $i \in \llbracket 0, M-1 \rrbracket$  do
3      $c_i += A_{i,k} \times b_k$ 
```

Algorithm 1.4: Dense Matrix-Matrix
Outer-Product Algorithm

```

1 for  $k \in \llbracket 0, K-1 \rrbracket$  do
2   for  $i \in \llbracket 0, M-1 \rrbracket$  do
3     for  $j \in \llbracket 0, N-1 \rrbracket$  do
4        $C_{i,j} += A_{i,k} \times B_{k,j}$ 
```

1.4.0.3 Gustavson’s Algorithm

As the MM algorithms have three ranks instead of two for MV, there is a third loop ordering, known as *Gustavson’s* algorithm [20] where the common rank K is the middle-loop (see line 2 in Algorithm 1.5). With this approach, the output is computed row-by-row with *PO*-rows generated and accumulated for each output C -row. With this algorithm, updates of the C -rows are sequential, but elements within the rows are updated with indirect accesses. In the

dense case, the B -matrix is entirely read M times and K PO -rows, of size N , are generated. In the context of sparse matrices, conversely to the C -rows, the selection of the rows in B is indirect, but the accesses are sequential within a row. Thus, the *intersection* search is only needed for the B -rows and not each element. The PO -rows can be accumulated before computing the next C -row. Again, for this algorithm, exchanging M and N causes the three matrices to be accessed *column-wise* instead of *row-wise*.

Algorithm 1.5: Dense Matrix-Matrix
Gustavson's Algorithm

```

1 for  $i \in \llbracket 0, M-1 \rrbracket$  do
2   for  $k \in \llbracket 0, K-1 \rrbracket$  do
3     for  $j \in \llbracket 0, N-1 \rrbracket$  do
4        $C_{i,j} += A_{i,k} \times B_{k,j}$ 

```

1.4.0.4 Comparison of the Algorithms

In Table 1.1, we summarize the main features of **IP**, **OP** and **Gust** in terms of accesses to the matrix A , B (or b) and C (or c). From this high level analysis, we can easily see that the data that is accessed with indirections shifts depending on the algorithm, from the input B (or b) to the output C (or c), with **Gust** being a trade-off between **IP** and **OP** for MM multiplication. To have a better understanding of the consequences on the actual number of memory accesses, we propose a more detailed analysis in Section 1.5.4, for the case of SpMSPM.

NOTE

Note that these algorithms are available in dense as well as in sparse MV and MM multiplications. In the sparse variants, the above algorithms (from 1.1 to 1.5) have the same behavior from a mathematical point-of-view, but need adjustments to traverse a sparse format instead of a dense matrix. Sparse versions of these algorithms are presented in Section 1.5.

Table 1.1: Comparison of Matrix Multiplication Algorithms

	Inner-Product (IP) ¹	Outer-Product (OP) ¹	Gustavson (Gust) ²
A (M, K) Accesses	• <i>row-wise</i> • read N times^{3,4} • sequential	• <i>column-wise</i> • read once • sequential	• <i>row-wise</i> • read once • sequential
B (K, N) Accesses	• <i>column-wise</i> • read M times⁵ • not sequential	• <i>row-wise</i>⁶ • read $M \times d(A)$ times^{4,7} • sequential	• <i>row-wise</i> • read $M \times d(A)$ times⁷ • sequential within rows
C (M, N) Accesses	• generated <i>row-wise</i>⁶ • one sequential traversal	• generated without specific structure • K POs produced⁸ (each PO has $\overline{nnz}_c(A) \times \overline{nnz}_r(B)$ non-zero elements⁹)	• generated <i>row-wise</i> • $K \times d(A)$ PO-rows produced¹⁰ for each row of A (M) ($\overline{nnz}_r(B)$ non-zero elements per PO-row)

¹ Considering $N = 1$ in the case of MV multiplication.

² Only for MM multiplication.

³ Or less than N times if B has empty columns.

⁴ Read once in the case of MV multiplication.

⁵ If B is stored with a sorted sparse format, else it would be read $\overline{nnz}_r(A) \times M = nnz(A)$ times.

⁶ *element-wise* in the case of MV multiplication (there is no row).

⁷ Which is equal to $\overline{nnz}_c(A)$, the average number of non-zero elements in the columns of A .

⁸ Or less than K times if A has empty columns or B has empty rows.

⁹ $\overline{nnz}_c(A)$ in the case of MV multiplication.

¹⁰ Which is equal to $\overline{nnz}_r(A)$, the average number of non-zero elements in the rows of A .

CONCLUSION

There are *three main algorithms* to compute sparse MV and MM kernels, **Inner-Product (IP)**, **Outer-Product (OP)** and **Gustavson (Gust)**. Each presents different characteristics, the main

one being the **irregularity of the accesses**: on the **input** matrix B (or vector b) with **IP**; on the **output** matrix C (or vector c) with **OP**; on both with **Gust** but at **row scale**.

1.4.1 Tiling

The *tiling* process [19] consists in partitioning a matrix: a matrix may be divided into *tiles*, non-overlapping sub-matrices. In other words, this method consists of adding ranks to delimit the *tiles*, and thus adding loop iterations in the kernel algorithm.

Tiling methods make it possible to process smaller portions of the matrix and thus to adapt the working set to the size of the local memory [31, 45], particularly when the matrix has a structure with denser regions. *Tiling* can also increase parallelism opportunities. Software optimizations for sparse matrix computation, such as OSKI [42] and Sparse BLAS [13], are based on *tiling*. In previous sections, the study of the **IP**, **OP** and **Gust** algorithms highlighted the influence of rank ordering on the sequentiality and data movement, and thus, adding intermediate ranks with *tiling* breaks the regularity of pure **IP** or **OP** algorithms, requiring additional hardware support (see Section 1.5).

For example, let us consider the A -matrix in Figure 1.5 which is divided into four *tiles* (one level of *tiling*). For a SpMV, two new ranks appear for a total of four ranks, as shown in Algorithm 1.6: M_1 and K_1 allow *tile* selection and M_2 and K_2 allow data access within a *tile*. Thus, we could decide to apply an **OP** on *tiles* and conversely an **IP** within *tiles*, mixing-up the pros and cons of these two algorithms at different rank levels. In the case of the example in Figure 1.5, the upper-left A -tile (red) is evaluated first, sequentially updating the upper tile of the c -vector in an **IP** manner. The lower-left A -tile (yellow) is then computed the same way. At this point, *intersections* have been done with only the upper b -tile, and c has been updated sequentially (first **PO**). The two next A -tiles (green, then blue) will compute *intersections* with the lower b -tile and update the entire c -vector sequentially, showing the **OP** characteristic of having **POs** to *merge*, two in this case.

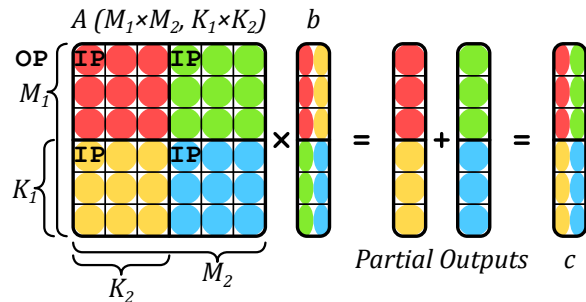


Figure 1.5: An Example of Tiling on SpMV

Algorithm 1.6: Matrix-Vector Algorithm with Custom Tiling of Figure 1.5

```

1 for  $k_1 \in [0, K_1 - 1]$  do
2   for  $i_1 \in [0, M_1 - 1]$  do
3     for  $i_2 \in [0, M_2 - 1]$  do
4       for  $k_2 \in [0, K_2 - 1]$  do
5          $i \leftarrow i_1 \times M_2 + i_2$ 
6          $k \leftarrow k_1 \times K_2 + k_2$ 
7          $c_i += A_{i,k} \times b_k$ 

```

DEFINITION

Tiling is a method for partitioning matrices in non-overlapping sub-matrices, called **tiles**.

NOTE

Note that in case of *tiling*, the A -matrix is no longer read sequentially, which can be problematic if it is stored in a classical sparse format like CSR that is specifically made for *row-wise* accesses. Custom sparse formats adapted for *tiling* can be used [22], but this requires pre-processing to perform the conversion.

CONCLUSION

Tiling methods are widely used to **distribute workloads** across multicore systems, and to **locally reduce the data dimensions**. Complex *tiling* algorithms with many possible rank ordering **inherit the behaviors of IP and OP**, and can be described as a combinations of both at different *tile* levels.

1.5 Hardware Challenges for Sparse Matrix Computation

IN the previous section we presented a brief background on matrix multiplication without considering the underlying hardware. Although the various sparse formats reduce the memory footprint, they have the disadvantage that indirect accesses are required, which creates data-dependencies and reduces the effectiveness of caches. As we have seen, with all three algorithms, at least some of the accesses are irregular. The lack of spatial locality for these accesses reduces the benefit of having caches [35, 47]. In other words, a full cache-line is loaded, but only a few entries are read or modified. Moreover, the matrices of interest are extremely large and even the non-zero elements, and associated metadata, can not generally fit in the cache. Depending on the algorithm, successive accesses to the same element may be too far apart to benefit from temporal locality in the cache.

1.5.1 Inner-Product Algorithm

In an **IP** algorithm, the A -metadata needs to be read to indirectly access the B -matrix (or b -vector), which may itself be stored in a sparse format. In this case, the algorithm needs to check if the non-zero A -indices are present in B (or b). If an index appears in neither list, there is no need to perform the multiplication (ineffectual operation). More generally, this problem applies to any *fiber* and is called the *intersection* search. As a minimum, this requires reading all the metadata for A and B (or b), and identifying those present in both.

In Algorithm 1.7, we present the pseudo-code for an *Inner-Product (IP)* SpMSpM with A and B stored in CSR and CSC formats, respectively. The outer-loop (M , line 1) iterates over the rows of A . The middle-loop (N , line 2) iterates over the columns of B . The inner-loop (line 4) iterates over the non-zero elements of the current row (i) of A , and for each index (k_A of position pos_A in the idx array of A , line 5), a search is performed in the metadata of B (line 6), to see if the corresponding index in B is present. Assuming it is ($isMatch$ is true, line 7), the partial sum is computed and then accumulated locally (acc , line 8). This example highlights the need to efficiently search for the *intersection* of indices in the metadata of the inputs.

Different algorithms can be used to address the *intersection* search, depending on whether the elements are sorted. When the elements are sorted, the *intersection* becomes a *merge* and generally requires at most $2 \times nnz - 1$ comparisons (in some cases the number of comparisons can be reduced [3]). In Algorithm 1.7, we use sorted sparse formats (CSR and CSC). Thus, the naïve *intersection* search proposed (line 10) does at most one traversal of each the current row of A (i) and the current column of B (j). Note that computing an *intersection* in software induces the use of conditional loops and branches, which can be costly for classical CPU branch predictors. As we see in Chapter ??, certain accelerators provide support for performing this *intersection* operation directly in hardware.

DEFINITION

The **intersection search** consists in founding non-zero elements of two sparse arrays that have matching indices. This search is costly in software on classical CPUs, thus being key to improve performance of **IP** algorithm.

1.5.2 Outer-Product Algorithm

In an **OP** algorithm, the inputs A and B (or b) are accessed sequentially, but the updates of the output C (or c) require indirect accesses. The pattern of the accesses, while the output is generated, is irregular and determined by the structure of the inputs. Furthermore, if these “random” updates provoke cache misses where only one word within a cache-line is modified, the bandwidth to external memory is poorly utilized. The **OP** algorithm generates several *partial outputs (POs)* that must be accumulated to obtain the final output. We can identify two strategies: *immediate update* and *deferred update*. In the former, the final output is immediately updated as each partial sum is generated. If the output is stored using a sparse format, either the index being updated does not yet exist, and the new partial sum must be inserted. Or, if it exists, a lookup must be done to locate it, the update performed and the result written back. The problem associated with combining the *POs* is called *reduction*.

In Algorithm 1.8, we present two pseudo-code for an *Outer-Product (OP)* SpMSpM using *immediate update* (from line 1) and *deferred update* (from line 7) with A and B stored in CSC and CSR formats, respectively. The outer-loop (K , line 1 or 7) iterates over the columns of A and the rows of B (common rank). The middle-loop (line 2 or 9) iterates over the non-zero

Algorithm 1.7: IP SpMSpM Performing *Intersection* Searches with CSR and CSC Formats

Data: $A(M, K)$ in CSR: $A.val, A.idx, A.ptr$
Data: $B(K, N)$ in CSC: $B.val, B.idx, B.ptr$
Data: $C(M, N)$ in dense format (for readability)
Result: $C = A \times B$

```

1 for  $i \in \llbracket 0, M-1 \rrbracket$  do                                     // Iterations over the rows of A
2   for  $j \in \llbracket 0, N-1 \rrbracket$  do                                     // Iterations over the columns of B
3      $acc \leftarrow 0; pos_B \leftarrow B.ptr_j;$ 
4     for  $pos_A \in \llbracket A.ptr_i, A.ptr_{i+1} - 1 \rrbracket$  do
5       // Iterations over the non-zero elements of the i-th row of A
6        $k_A \leftarrow A.idx_{pos_A};$ 
7       // Column-index of the (i, pos_A) non-zero element of A
8        $(isMatch, pos_B) \leftarrow IntersectionSearch(k_A, j, pos_B, B.idx, B.ptr);$ 
9       // Return true and the matching pos_B if idx exists in the j-th column of B
10      if  $isMatch$  then                                           // If indices did match
11         $acc \leftarrow acc + A.val_{pos_A} \times B.val_{pos_B};$ 
12        // Local partial sum accumulation
13       $C_{i,j} \leftarrow acc;$                                      // Write in memory of the resulting (i, j) element of C
14
15 function  $IntersectionSearch(k_A, j, pos_B, B.idx, B.ptr)$  is
16   // Naive intersection search allowing the i-th row of A and the j-th column of B to
17   // be traversed only once at iteration (i, j)
18   while  $B.idx_{pos_B} < k_A$  and  $pos_B < B.ptr_{j+1}$  do
19      $pos_B \leftarrow pos_B + 1;$ 
20   if  $B.idx_{pos_B} = k_A$  then                                     // Returns true if indices from A and B match
21     return  $(true, pos_B);$ 
22   return  $(false, pos_B);$ 

```

elements of the current column (k) of A . The inner-loop (line 4 or 11) iterates over the non-zero elements of the current row (k) of B . Because we only iterate over the non-zero values, there is no need for an *intersection* search. The partial sum is then computed (line 6 or 15). Finally, in the *immediate update* version, the partial sum must be accumulated to C (line 6). These accesses are irregular, making it necessary to lookup the value in C , update it, and write it back. This is a *reduction* operation, performed on irregular data.

To avoid the complexities of these “random” *reductions* with *immediate updates*, the *deferred update* version postpones the accumulation and simply stores the partial sums in memory. One such technique is called *output batching* [28]. With this technique, arrays containing the indices and the partial sum are streamed sequentially to memory (PO_k , lines 13-15). Then, during a second pass (line 17), they are read back and the accumulations are performed, an approach which results in sequential memory accesses during both the first and second passes, with the downside that the volume of data transferred to memory is increased. As we see in Chapter ??, hardware accelerators may combine the *immediate* and *deferred update* techniques to optimize memory bandwidth.

DEFINITION

“Random” reductions occur when updating data from memory at irregular addresses, which happen on *immediate update* OP. With *deferred update* OP, the *reductions* are sequential but large sets of data need to be buffered in memory. Optimizing *reduction* operations is key to improving the performance on OP algorithm.

1.5.3 Summary

In Table 1.2, we summarize the advantages and hardware challenges for each algorithm, including **Gust** which presents a trade-off between **IP** and **OP**. The potential for parallelism and data-reuse varies across algorithms, **IP** being the most limited unless *tiling* methods are used. With **OP**, PO s can easily be generated across a multicore system but at the cost of transferring a large volume of data to memory. With *immediate updates*, **OP** transfers less data but all accesses on the output are “random” *reductions*.

The issues presented with **IP** and **OP** can be generalized to two concepts: *gather* and

Algorithm 1.8: *op* SpMSpM Performing *Reduction* with CSR and CSC Formats

Data: $A(M, K)$ in CSR: $A.val, A.idx, A.ptr$
Data: $B(K, N)$ in CSC: $B.val, B.idx, B.ptr$
Data: $PO_k(M, N)$ for all $k \in \llbracket 0, K-1 \rrbracket$ in COO: $PO_k.val, PO_k.row, PO_k.col$
Data: $C(M, N)$ in dense format (for readability)
Result: $C = A \times B$

```

// * * * * *
// * * * 'Immediate updates' version * * * * *
1 for  $k \in \llbracket 0, K-1 \rrbracket$  do // Iterations over the columns of A and the rows of B
2   for  $pos_A \in \llbracket A.ptr_k, A.ptr_{k+1} \rrbracket$  do
3     // Iterations over the non-zero elements of the k-th column of A
4      $i_A \leftarrow A.idx_{pos_A};$  // Row-index of the ( $pos_A$ ,  $k$ ) non-zero element of A
5     for  $pos_B \in \llbracket B.ptr_k, B.ptr_{k+1} \rrbracket$  do
6       // Iterations over the non-zero elements of the k-th row of B
7        $j_B \leftarrow B.idx_{pos_B};$  // Column-index of the ( $k$ ,  $pos_B$ ) non-zero element of B
8        $C_{i_A, j_B} \leftarrow C_{i_A, j_B} + A.val_{pos_A} \times B.val_{pos_B};$ 
9       // Reduction (to memory) of the partial sum into the ( $i_A$ ,  $j_B$ ) element of C
10      (immediate update)
// * * * * *
// * * * 'Deferred updates' version * * * * *
// Multiply phase
7 for  $k \in \llbracket 0, K-1 \rrbracket$  do // Iterations over the columns of A and the rows of B
8    $pos_{PO} \leftarrow 0;$ 
9   for  $pos_A \in \llbracket A.ptr_k, A.ptr_{k+1} \rrbracket$  do
10    // Iterations over the non-zero elements of the k-th column of A
11     $i_A \leftarrow A.idx_{pos_A};$  // Row-index of the ( $pos_A$ ,  $k$ ) non-zero element of A
12    for  $pos_B \in \llbracket B.ptr_k, B.ptr_{k+1} \rrbracket$  do
13      // Iterations over the non-zero elements of the k-th row of B
14       $j_B \leftarrow B.idx_{pos_B};$  // Column-index of the ( $k$ ,  $pos_B$ ) non-zero element of B
15       $PO_k.row_{pos_{PO}} \leftarrow i_A;$ 
16       $PO_k.col_{pos_{PO}} \leftarrow j_B;$ 
17       $PO_k.val_{pos_{PO}} \leftarrow A.val_{pos_A} \times B.val_{pos_B};$ 
18      // The k-th partial sum of the ( $i_A$ ,  $j_B$ ) element of C is stored in memory in the
19      k-th partial output
20       $pos_{PO} \leftarrow pos_{PO} + 1;$ 
21      // Each partial output is generated sorted, row-wise
// Merge phase
17 for  $k \in \llbracket 0, K-2 \rrbracket$  do // Naive serial merge of the K partial outputs
18    $PO_{k+1} \leftarrow 2DListsMerge(PO_k, PO_{k+1});$ 
19   //  $PO_{K-1}$  is C stored in COO format
20   for  $pos \in \llbracket 0, length(PO_{K-1}) - 1 \rrbracket$  do
21     // Update C (dense) from the fully merged partial outputs
22      $i \leftarrow PO_{K-1}.row_{pos};$ 
23      $j \leftarrow PO_{K-1}.col_{pos};$ 
24      $C_{i, j} \leftarrow PO_{K-1}.val_{pos};$  // Write in memory of the ( $i$ ,  $j$ ) element of C

```

Table 1.2: Comparison of Advantages and Challenges with Matrix Multiplication Algorithms

	<i>Inner-Product (IP)</i>	<i>Outer-Product (OP)</i>	<i>Gustavson (Gust)</i>
Access Count	MV A: once b: $\text{nnz}(A) (\sim M \times)$ c: once MM A: $N \times$ B: $M \times$ C: once	A: once b: once c: $\text{nnz}(A) (\sim K \times)$ A: once B: $M \times d(A) \times$ C: $\text{nnz}(A) \times \overline{\text{nnz}}_r(B) (\sim K \times)$	N/A A: once B: $M \times d(A) \times$ C: $\text{nnz}(A) \times \overline{\text{nnz}}_r(B)$
Sequential Accesses	Accesses to A and C	Accesses to A and B	Accesses to A and C Accesses to B -rows
Input Format	Accesses to A and B benefit from alternate formats (CSR and CSC)	Accesses to A and B benefit from alternate formats (CSR and CSC)	Allows consistent format for A and B
Parallelism Opportunity	Possible with <i>tiling</i>	Generation of PO s over common rank of A and B	Reading A -rows and updating C -rows
Reuse Opportunity	Potentially B , but limited by its size	An A -column or B -row while computing PO s	A set of B -rows, based on non-zero elements of A -rows
Challenges	<i>Intersection</i> search between non-zero elements in A and B	Storage for PO s or in-place <i>merge</i> and <i>reduce</i>	<i>Merge</i> and <i>reduce</i> of PO -rows

scatter. The *gather* problem exists when the inputs are indirectly accessed, and need to be read from memory using metadata to present the processor with the correct terms for computing the partial sums. Typically, with **IP**, the *gather* problem is more difficult. The *scatter* problem exists when the order in which the output matrix/vector is generated is not sequential and is typically associated with the **OP** algorithm. In the case of *tiling*, both *gather* and *scatter* must be performed creating a need for efficient *intersection* searches and random *reductions*.

CONCLUSION

There are **two main challenges** to address when computing sparse matrices. When reading irregular data from memory, addressing the ***gather*** problem is key. It consists of processing efficient ***intersection searches***. When updating irregular data from memory, addressing the ***scatter*** problem is key. It consists of ***managing POs*** in memory or processing efficient ***“random” reductions***. These two challenges are tied to **IP** an **OP**, respectively, and are present in any higher rank algorithms for sparse matrix computing.

1.5.4 SpMSpM Algorithm Analysis

To better understand how the algorithms impact the number and types of memory accesses, we developed, and presented in Figure 1.6, a model based on memory accesses counts. We identify five memory access patterns: (1) those where a matrix is read only once, thus there is no opportunity for reuse, (2) where the accesses are sequential, but a *fiber* is read multiple times, (3) where *fibers* are read consecutively, but the accesses within a *fiber* are “random”, (4) where the *fibers* are accessed “randomly”, but within a *fiber* the accesses are sequential, and (5) where the matrix elements are accessed multiple times, and with a “random” access pattern.

For each of the three algorithms, we counted the number of read and write accesses, based on the density of the input matrices, and the local memory storage available (“0D”, “1D” and “2D”), as presented in Table 1.3. We refer to a system with no local storage or cache as ‘0 Dimensional’ (“0D”), a system which has local storage that is sufficient to hold a single row or column as ‘1 Dimensional’ (“1D”), and a system which can cache an entire matrix as ‘2 Dimensional’ (“2D”). In Figure 1.6, we plotted the number of memory accesses for SpMSpM, in a logarithmic scale, based on a square matrix with $M = 10,000$. Indeed, the number of non-zero elements in C depends on the structure of the input matrices and in this model, we have assumed the non-zero entries in the input matrices are uniformly, randomly distributed. This corresponds to a *worst case* for sparse matrices (no spatial and temporal locality). If A or B are banded matrices, for example, there would be fewer non-zero elements in C . In each

graph, we have separated the accesses into A (bottom), B (middle) and C (top). The color code shows the access pattern, based on the five categories described above.

Table 1.3: Memory Access Count for Each Matrices of **IP**, **OP** and **Gust** SpMSPM

	Inner-Product (IP)	Outer-Product (OP)	Gustavson (Gust)
A (M, K)	OD: $N \times nnz(A)$ 1D-2D: $nnz(A)$	OD-1D-2D: $nnz(A)$	OD-1D-2D: $nnz(A)$
B (K, N)	OD-1D: $M \times nnz(B)$ 2D: $nnz(B)$	OD: $M \times d(A) \times nnz(B)$ 1D-2D: $nnz(B)$	OD-1D: $M \times d(A) \times nnz(B)$ 2D: $nnz(B)$
C (M, N)	OD-1D-2D: $nnz(C)$ (max: $M \times N$) ¹	OD-1D: $2 \times M \times d(A) \times nnz(B)$ 2D: $nnz(C)$ (max: $M \times N$) ¹	OD: $2 \times M \times d(A) \times nnz(B)$ 1D-2D: $nnz(C)$ (max: $M \times N$) ¹

¹ We can not easily quantify $nnz(C)$ knowing $nnz(A)$ and $nnz(B)$. We thus use a statistic approximation.

In the top-row of Figure 1.6 (graphs ①, ② and ③), we start by considering a naïve system which has no local storage (“OD”), thus all data comes from main memory. From the graphs in this row, we clearly see that **IP** requires more accesses at low density, as the A -matrix must be read N times (graph ①). Whereas for **OP** and **Gust**, the A -matrix is read only once (thus the *grey* color). For **OP** and **Gust** the total number of accesses is the same, however, with **OP** (graph ②), the accesses to C are “random” (*red*). With **Gust** (graph ③), the accesses to B are sequential within the rows (*orange*) and the C -rows are generated sequentially (*yellow*), thus **Gust** avoids having any fully “random” accesses (*red*).

Since main memory accesses are the principal bottleneck, systems integrate local storage (such as buffers or caches). Thus, we consider the case of a system with sufficient local memory to store one full row, per matrix (equivalent to few sparse rows, “1D”). We assume this local storage is perfect (no misses) and we re-calculate the number of remaining main memory accesses, as shown in the middle-row of Figure 1.6 (graphs ④, ⑤ and ⑥). The matrices are assumed to be initially stored in main memory, thus they must be accessed at least once. As can be seen, for **IP** (graph ④), the number of A -accesses is reduced, because rows are reused, thus the total number of A -accesses becomes equal to that in **OP** and **Gust**. For **OP** (graph ⑤), the single row buffer greatly reduces the number of B -accesses, however, it does nothing for C (despite the appearance, due to the logarithmic scale). For **Gust** (graph ⑥), the single row buffer is sufficient to cache the *reduction* operations in C , and thus, it now has fewer overall accesses than **OP**. It is also interesting to note that **OP** and **Gust** scale better than **IP** with lower densities, as seen by the shallower slope.

Finally, we have considered the extreme case where an system has a buffer of sufficient size to store two sparse matrices (“2D”), which is shown in the bottom-row of Figure 1.6 (graphs ⑦, ⑧ and ⑨). Although this case is not practical for large matrices, through the proper use of *tiling* to locally reduce the dimensions of a sub-matrix, this case can be achieved at the level of a *tile*. In this case, the total number of external accesses is equal for the three algorithms, as each matrix is accessed only once from main memory. For **IP** (graph ⑦), it is interesting to note that such a large buffer is needed to locally address the *intersection* search in B . Similarly for **OP** (graph ⑧), a large storage is required to address the *reductions* in C . For **Gust**, in fact, it is not necessary to buffer the entire B -matrix. Indeed, buffering a few rows (corresponding to $nnz_r(A)$) is sufficient to achieve the number of accesses shown in graph ⑨. This corresponds to an intermediate design point (multiple row buffers for B) not explicitly shown in the figure (between graphs ⑥ and ⑨).

When viewed overall, Figure 1.6 highlights the benefits of **Gust**. We clearly see that **Gust** does not require any fully “random” accesses (*red*). With a single row buffer, it is possible to address the *reductions* in C . And, with a limited number of row buffers, the number of external memory accesses for B can be reduced. Based on this analysis, it is clear that, among the three base algorithms, **Gust** is the best candidate for hardware implementation of SpMSPM.

CONCLUSION

Based on a high level analysis of the memory accesses on SpMSPM, **Gustavson’s algorithm** appears as the **best trade-off** to minimize memory traffic with reasonable amount of local storage. The second best option is **OP**, which requires fewer memory accesses than **IP** when the matrices have a low density.

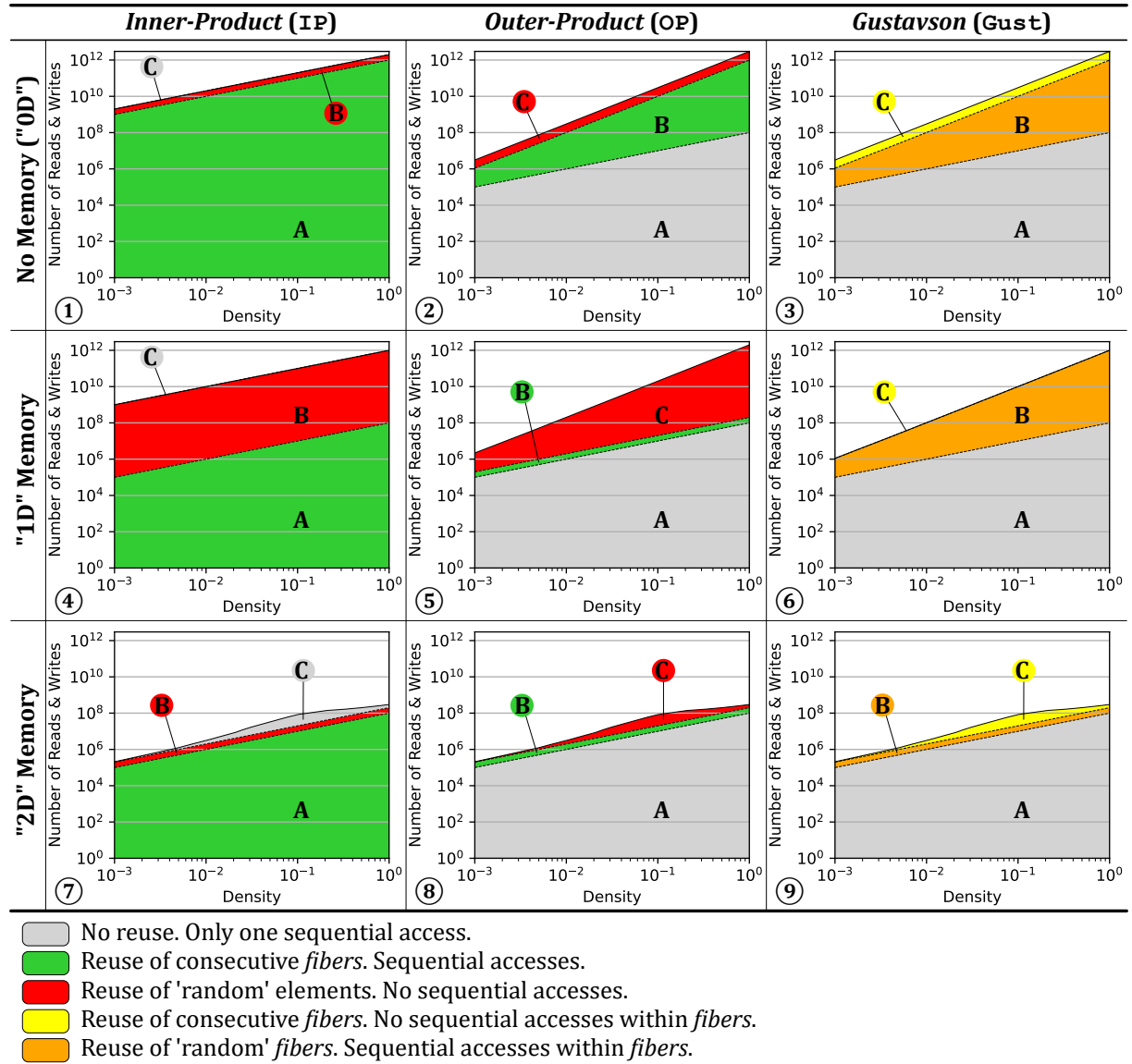


Figure 1.6: Model of Memory Access Counts Depending on Algorithm and Local Memory Size

1.6 Conclusion

IN this chapter, we established the foundation for understanding the relationship between applications working with irregular data and the underlying challenges in hardware. Irregular accesses are common in sparse computing and cause reduced cache efficiency, limited parallelism, and higher memory latency exposure. We analysed the most representative algorithms for sparse matrix multiplication kernels, and identified their key operations that suffer from irregularities. *Intersections* and *reductions* on irregular data with classical systems are costly, thus being the central challenges of sparse computation, and driving motivation for designing specialized hardware as we show in the next chapter.

TAKEAWAYS

Sparse matrices: large, highly sparse, structured (such as banded matrices)

Sparse formats: diverse, reduce memory footprint

- COO, the simplest
- CSR and CSC, the mostly used
- BaCSC, our own sparse format for banded matrices

Sparse kernels: sparse MV and MM multiplications (such as SpMV and SpMSPM), generate indirect accesses

Algorithms: *Inner-Product (IP)*, *Outer-Product (OP)* and *Gustavson (Gust)*, extended with *tiling* methods

- Best potential to reduce memory transfers: **Gust**, followed by **OP** at low density

Problem: irregular accesses, generate high memory transaction latencies

Challenges:

- *gather*: *intersection* searches
- *scatter*: “random” *reductions*, or storing and *merging partial outputs (POs)*

Conclusion

Contents

1	TODO	28
---	----------------	-----------

1 TODO

TODO

Appendices

Contents

A	Sparse Formats	30
B	A Generic Representation for <i>Sparse Formats</i>	31
C	Extended Overview of the State-of-the-Art Accelerators for Sparse Computing	32
D	Analytical Model of a <i>Reduction Cache</i> Computing an OP SpMV Kernel	33
E	Python Model of a Data-Cache with Support for <i>Reductions</i>	34
F	Statistics	35

A Sparse Formats

TODO

COO The simplest *sparse format* is called *COO* (*COOrdinate*) [11, 39, 41]. It consists in three separate tables, one filled only with the non-zero elements of the matrix, a row index table containing the row of the corresponding elements, and similarly a column index table (see Figure ??). The two latter are *metadata* tables and all three tables are of size nnz . In this format, the elements are not necessarily sorted, the memory footprint does not depend on the structure of the matrix and all elements are stored independently of each other.

CSR & CSC The most commonly used *sparse format* is called *CSR* (*Compressed Sparse Row*) [4, 11, 39, 41]. It is similar to the *COO* format except that the row index table is replaced by a set of indices that point to the position of the start of each row in the column index table (see Figure ??). The size of this table is the number of rows plus one, and, in practical cases, the memory footprint is smaller than with *COO*. However, the non-zero elements need to be sorted in *row-wise* order. This format has a *column-wise* variant called *CSC* (*Compressed Sparse Column*) where the column index table stores the positions of columns instead of rows. These two formats are symmetrical but with *CSR*, it is the traversal of rows that is efficient because they are sequential whereas in *CSC*, column traversal is efficient.

BCSR Even in sparse matrices, there can be regions, or blocks, that have a large fraction of non-zero values. In this case, the *BCSR* format [4], which is an extension of the *CSR* format with *tiling*, can be attractive. We use the term *tiling* to describe a level of hierarchy in a storage format. That is, a *tile* refers to a sub-matrix, which itself may be dense, as in the case of *BCSR*, or which could be sparse. With the *BCSR* format, we simply replace the non-zero elements of the *CSR* format with dense, fixed-size sub-matrices (see Figure ??). Within these sub-matrices, it is possible to have some zero elements. The advantage of the *BCSR* format is that the dense sub-matrices can be processed using techniques optimized for dense matrices, particularly using vector units or GPUs [22]. Compared to the *CSR* format, *BCSR* makes it possible to accelerate the computation within the dense *tiles*, but this comes with the overhead of storing some zero values. Not all matrices are well suited to the *BCSR* format.

DIA For diagonally structured matrices ($\forall (i, j) \in \llbracket 0, M \rrbracket \times \llbracket 0, N \rrbracket, a_{i,j} = 0 \text{ for } |i - j| > K$), a format called *DIA* (*DIAGonal*) is particularly well suited [4, 11, 39]. It consists in storing the values along the $2 \times K + 1$ diagonals in a table and keeping information about the position of the diagonals in the matrix (see Figure ??). The organization of the non-zero data and selection of the diagonals can be tailored to the matrices and access patterns. This format is well suited to diagonal matrices, has a low overhead for storing indices and can be combined with other formats to better fit more complex structures.

ELL Another format called *ELL* (*ELLPACK*) [4, 11, 27, 39] is well suited when the number of non-zero elements per row can be bounded by K . In this case, the non-zero values in a $M \times N$ matrix can be stored as a dense $M \times K$ matrix, augmented with an index table, of the same dimensions, which gives the column position of each non-zero value (see Figure ??). This format is not efficient if the number of non-zero elements per row is highly variable, as many entries in the $M \times K$ table are unused. This format is well suited to GPUs [9, 43] since it transforms a sparse matrix into a dense one in memory.

HYB The *HYB* format (*HYBrid*) [5] is an example of a format which combines *ELL* and *COO*. The *ELL* format can be used, but with a value of K (the width of the value table) that is smaller than the maximum non-zero entries per row in the original matrix (see Figure ??). For the small number of rows where the table lacks a sufficient number of columns, the *COO* format can be used to store the extra values. In this way, the storage efficiency of *ELL* is improved, and it can be applied to a broader class of matrices.

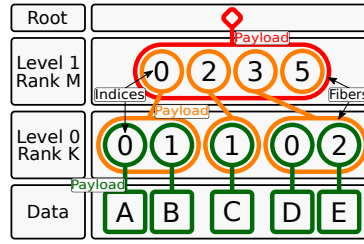


Figure 1.7: Example of the *FiberTree* Representation with CSR Format

B A Generic Representation for Sparse Formats

A visual representation of compressed matrices, called *FiberTree* [40], is helpful to understand these sparse formats. Each *rank* of a matrix represents a level of a tree where the nodes are filled with either the metadata of the child nodes or the matrix data which can only appear in the leaves (see Figure 1.7). With this representation, we clearly identify the number of *ranks*, which define the *fibers*. A *fiber* is the generic term to describe a one-dimensional stream of data or metadata, which corresponds to a row or column in the case of a two-dimensional matrix. A stream of indices is a *fiber* of metadata. Figure 1.7 presents, as an example, a *FiberTree* representation of the matrix of Figure ??, matching the CSR format: each *rank* in the tree corresponds to a *fiber* stored in the table *Row PTR* or *Col IDX* of Figure 1.3b.

In a recent work [46], a systematic nomenclature for *sparse matrix (and tensor) formats* was proposed, based on *FiberTree* representation. In fact, in the example of Figure 1.7, the *fibers* are compressed with different methods. Thus each *rank* can be characterized by a label indicating how it is compressed: *Uncompressed (U)*, *Coordinate Payload (CP)*, *Uncompressed Offset Pairs (UOP)*. *U* defines *ranks* where all data values are stored contiguously in memory. *CP* defines *ranks* that store the indices of each payload, a payload being either a non-zero element or a sub-matrix¹. *UOP* defines *ranks* that store pointers (offsets) to delimit sub-matrices of the next *rank*². These are the three main ones as they are used in the most common sparse formats but many others exist or can be created.

With this nomenclature, the sparse formats can be classified according to their *rank* compression methods. For example, the CSR format is denoted as *(UOP, CP) row-major* since its *Row PTR* table stores pointers (offsets) to rows. The CSR and CSC formats are both of type *(UOP, CP)*, one being *row-major*, the other *column-major*. The COO format is denoted as *CP²* because there are two coordinates (row and column) for each non-zero element, as it is a *CP fiber* having two indices per data element instead of one. User defined formats can also be easily described and named. For example, a user could define a storage format with a nested CSR. That is, each non-zero element in the outer CSR, corresponds to a sub-matrix, itself stored in CSR format. Such a representation would be labeled *(UOP, CP, UOP, CP)*. A similar nomenclature is proposed by TACO [11]. It is more adapted to code generation applications, where the *FiberTree*-based nomenclature better describes the memory storage of the sparse format. For example, *U* corresponds to *Dense*, *CP* to *Singleton*, the pair *(UOP, CP)* is translated as *Compressed* while *UOP* alone corresponds to *Offset*. We chose to adopt the *FiberTree*-based nomenclature in this article since it is more suited to a hardware context.

¹In this section, we use the term sub-matrix but this is extended to the notion of *tiling* in Section 1.4.1.

²If some sub-matrices are empty, pointers will be repeated, hence 'uncompressed'.

C Extended Overview of the State-of-the-Art Accelerators for Sparse Computing

TODO

D Analytical Model of a *Reduction Cache* Computing an OP SpMV Kernel

TODO

E Python Model of a Data-Cache with Support for *Reductions*

TODO

F Statistics

TODO (get infos before end of contract)
(commits, nb. of lines, bugs, timeline -> into figures?)

List of Figures

1.1	Examples of Banded Matrix Structure	15
1.2	Banded Matrix Definition	15
1.3	Example of a Sparse Matrix in COO, CSR and CSC Formats	16
1.4	Example of a Sparse Matrix in BaCSC Format	17
1.5	An Example of <i>Tiling</i> on SpMV	20
1.6	Model of Memory Access Counts Depending on Algorithm and Local Memory Size	25
1.7	Example of the <i>FiberTree</i> Representation with CSR Format	31

List of Tables

1.1	Comparison of Matrix Multiplication Algorithms	19
1.2	Comparison of Advantages and Challenges with Matrix Multiplication Algorithms	24
1.3	Memory Access Count for Each Matrices of IP , OP and Gust SpMSPM	25

List of Algorithms

1.1	Dense Matrix-Vector <i>Inner-Product</i> Algorithm	18
1.2	Dense Matrix-Matrix <i>Inner-Product</i> Algorithm	18
1.3	Dense Matrix-Vector <i>Outer-Product</i> Algorithm	18
1.4	Dense Matrix-Matrix <i>Outer-Product</i> Algorithm	18
1.5	Dense Matrix-Matrix <i>Gustavson's</i> Algorithm	19
1.6	Matrix-Vector Algorithm with Custom <i>Tiling</i> of Figure 1.5	20
1.7	IP SpMSpM Performing <i>Intersection</i> Searches with CSR and CSC Formats	22
1.8	OP SpMSpM Performing <i>Reduction</i> with CSR and CSC Formats	23

List of Source Code

Bibliography

- [1] Venkitesh Ayyar, Evan Weinberg, Richard C. Brower, M.A. Clark, and Mathias Wagner. 2023. Optimizing Staggered Multigrid for Exascale performance. In *Proceedings of The 39th International Symposium on Lattice Field Theory — PoS(LATTICE2022)*, Vol. 430. 335. <https://doi.org/10.22323/1.430.0335>
- [2] Ariful Azad, Aydin Buluç, and John Gilbert. 2015. Parallel Triangle Counting and Enumeration Using Matrix Algebra. In *2015 IEEE International Parallel and Distributed Processing Symposium Workshop*. 804–811. <https://doi.org/10.1109/IPDPSW.2015.75>
- [3] Ricardo Baeza-Yates and Alejandro Salinger. 2010. *Fast Intersection Algorithms for Sorted Sequences*. Springer Berlin Heidelberg, Berlin, Heidelberg, 45–61. https://doi.org/10.1007/978-3-642-12476-1_3
- [4] Richard Barrett, Michael Berry, Tony F. Chan, James Demmel, June Donato, Jack Dongarra, Victor Eijkhout, Roldan Pozo, Charles Romine, and Henk van der Vorst. 1994. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. Society for Industrial and Applied Mathematics. <https://doi.org/10.1137/1.9781611971538>
- [5] Nathan Bell and Michael Garland. 2009. Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis* (Portland, Oregon) (SC '09). Association for Computing Machinery, New York, NY, USA, Article 18, 11 pages. <https://doi.org/10.1145/1654059.1654078>
- [6] Achi Brandt and Dorit Ron. 2003. *Multigrid Solvers and Multilevel Optimization Strategies*. Springer US, Boston, MA, 1–69. https://doi.org/10.1007/978-1-4757-3748-6_1
- [7] Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems* 30, 1 (1998), 107–117. [https://doi.org/10.1016/S0169-7552\(98\)00110-X](https://doi.org/10.1016/S0169-7552(98)00110-X) Proceedings of the Seventh International World Wide Web Conference.
- [8] Aydın Buluç, John Gilbert, and Viral B. Shah. 2011. 13. *Implementing Sparse Matrices for Graph Algorithms*. Society for Industrial and Applied Mathematics, Chapter 13, 287–313. <https://doi.org/10.1137/1.9780898719918.ch13>
- [9] Wei Cao, Lu Yao, Zongzhe Li, Yongxian Wang, and Zhenghua Wang. 2010. Implementing Sparse Matrix-Vector multiplication using CUDA based on a hybrid sparse matrix format. In *2010 International Conference on Computer Application and System Modeling (ICCSM 2010)*, Vol. 11. V11–161–V11–165. <https://doi.org/10.1109/ICCSM.2010.5623237>
- [10] Timothy M. Chan. 2007. More Algorithms for All-Pairs Shortest Paths in Weighted Graphs. In *Proceedings of the Thirty-Ninth Annual ACM Symposium on Theory of Computing* (San Diego, California, USA) (STOC '07). Association for Computing Machinery, New York, NY, USA, 590–598. <https://doi.org/10.1145/1250790.1250877>
- [11] Stephen Chou, Fredrik Kjolstad, and Saman Amarasinghe. 2018. Format Abstraction for Sparse Tensor Algebra Compilers. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 123 (oct 2018), 30 pages. <https://doi.org/10.1145/3276493>
- [12] Stijn Dongen. 2000. Graph Clustering by Flow Simulation. *PhD thesis, Center for Math and Computer Science (CWI)* (05 2000).

- [13] Iain S. Duff, Michael A. Heroux, and Roldan Pozo. 2002. An Overview of the Sparse Basic Linear Algebra Subprograms: The New Standard from the BLAS Technical Forum. *ACM Trans. Math. Softw.* 28, 2 (jun 2002), 239–267. <https://doi.org/10.1145/567806.567810>
- [14] D. K. Faddeev and V. N. Faddeeva. 1981. Computational methods of linear algebra. *Journal of Soviet Mathematics* 15, 5 (1981), 531–650. <https://doi.org/10.1007/BF01086544>
- [15] Mirko Farina, Usman Ahmad, Ahmad Taha, Hussein Younes, Yusuf Mesbah, Xiao Yu, and Witold Pedrycz. 2024. Sparsity in transformers: A systematic literature review. *Neurocomputing* 582 (2024), 127468. <https://doi.org/10.1016/j.neucom.2024.127468>
- [16] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Atlanta, Georgia) (SC '20). IEEE Press, Article 17, 14 pages. <https://dl.acm.org/doi/10.5555/3433701.3433723>
- [17] John R. Gilbert, Steve Reinhardt, and Viral B. Shah. 2006. High-Performance Graph Algorithms from Parallel Sparse Matrices. In *Proceedings of the 8th International Conference on Applied Parallel Computing: State of the Art in Scientific Computing* (Umeå, Sweden) (PARA'06). Springer-Verlag, Berlin, Heidelberg, 260–269. https://link.springer.com/chapter/10.1007/978-3-540-75755-9_32
- [18] John R. Gilbert, Steve Reinhardt, and Viral B. Shah. 2008. A Unified Framework for Numerical and Combinatorial Computing. *Computing in Science & Engineering* 10, 2 (March 2008), 20–25. <https://doi.org/10.1109/MCSE.2008.45>
- [19] Branko Grünbaum and Geoffrey C. Shephard. 1987. *Tilings and Patterns*. W. H. Freeman & Co., New York.
- [20] Fred G. Gustavson. 1978. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Trans. Math. Softw.* 4, 3 (sep 1978), 250–269. <https://doi.org/10.1145/355791.355796>
- [21] Václav Hapla, David Horák, and Michal Merta. 2012. Use of Direct Solvers in TFETI Massively Parallel Implementation. In *Proceedings of the 11th International Conference on Applied Parallel and Scientific Computing* (Helsinki, Finland) (PARA'12). Springer-Verlag, Berlin, Heidelberg, 192–205. https://doi.org/10.1007/978-3-642-36803-5_14
- [22] Eun-Jin Im, Katherine Yelick, and Richard Vuduc. 2004. Sparsity: Optimization Framework for Sparse Matrix Kernels. *The International Journal of High Performance Computing Applications* 18, 1 (2004), 135–158. <https://doi.org/10.1177/1094342004041296> arXiv:<https://doi.org/10.1177/1094342004041296>
- [23] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. <https://doi.org/10.1145/3640542>
- [24] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–7. <https://doi.org/10.1109/ASAP61560.2024.00012>
- [25] Satoshi Itoh, Pablo Ordejón, and Richard M. Martin. 1995. Order-N tight-binding molecular dynamics on parallel computers. *Computer Physics Communications* 88, 2 (1995), 173–185. [https://doi.org/10.1016/0010-4655\(95\)00031-A](https://doi.org/10.1016/0010-4655(95)00031-A)
- [26] Jeremy Kepner, David Bader, Aydın Buluç, John Gilbert, Timothy Mattson, and Henning Meyerhenke. 2015. Graphs, Matrices, and the GraphBLAS: Seven Good Reasons. *Procedia Computer Science* 51, International Conference On Computational Science, ICCS 2015 (2015), 2453–2462. <https://doi.org/10.1016/j.procs.2015.05.353>
- [27] David R. Kincaid, Thomas C. Oppe, and David M. Young. 1989. *ITPACKV 2D user's guide*. Technical Report. United States. <https://doi.org/10.2172/7093021>

- [28] Vladimir Kiriansky, Yunming Zhang, and Saman Amarasinghe. 2016. Optimizing Indirect Memory References with Milk. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation* (Haifa, Israel) (PACT '16). Association for Computing Machinery, New York, NY, USA, 299–312. <https://doi.org/10.1145/2967938.2967948>
- [29] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 77 (oct 2017), 29 pages. <https://doi.org/10.1145/3133901>
- [30] Scott P. Kolodziej, Mohsen Aznaveh, Matthew Bullock, Jarrett David, Timothy A. Davis, Matthew Henderson, Yifan Hu, and Read Sandstrom. 2019. The SuiteSparse Matrix Collection Website Interface. *Journal of Open Source Software* 4, 35 (2019), 1244. <https://doi.org/10.21105/joss.01244>
- [31] Monica D. Lam, Edward E. Rothberg, and Michael E. Wolf. 1991. The Cache Performance and Optimizations of Blocked Algorithms. *SIGPLAN Not.* 26, 4 (apr 1991), 63–74. <https://doi.org/10.1145/106973.106981>
- [32] Ruipeng Li, Björn Sjögreen, and Ulrike Meier Yang. 2021. A New Class of AMG Interpolation Methods Based on Matrix-Matrix Multiplications. *SIAM Journal on Scientific Computing* 43, 5 (2021), S540–S564. <https://doi.org/10.1137/20M134931X>
- [33] J.-C. Luo. 1992. Algorithms for Reducing the Bandwidth and Profile of a Sparse Matrix. *Computers & Structures* 44, 3 (1992), 535–548. [https://doi.org/10.1016/0045-7949\(92\)90386-E](https://doi.org/10.1016/0045-7949(92)90386-E)
- [34] Liviu Maftciu-Scai. 2014. The Bandwidths of a Matrix. A Survey of Algorithms. *West University of Timisoara Annals, Timisoara, Romania* LII (12 2014), 183– 223. <https://doi.org/10.2478/awutm-2014-0019>
- [35] Kiran Matam, Siva Rama Krishna Bharadwaj Indarapu, and Kishore Kothapalli. 2012. Sparse matrix-matrix multiplication on modern architectures. In *2012 19th International Conference on High Performance Computing*. 1–10. <https://doi.org/10.1109/HiPC.2012.6507483>
- [36] Gerald Penn. 2006. Efficient transitive closure of sparse matrices over closed semirings. *Theoretical Computer Science* 354, 1 (2006), 72–81. <https://doi.org/10.1016/j.tcs.2005.11.008> Algebraic Methods in Language Processing.
- [37] Michael O Rabin and Vijay V Vazirani. 1989. Maximum matchings in general graphs through randomization. *Journal of Algorithms* 10, 4 (1989), 557–567. [https://doi.org/10.1016/0196-6774\(89\)90005-9](https://doi.org/10.1016/0196-6774(89)90005-9)
- [38] Oleg I. Ryabkov. 2022. Implementation of the Algebraic Multigrid Solver Designed for Graphics Processing Units Based on the AMGCL Framework. In *Parallel Computational Technologies*, Leonid Sokolinsky and Mikhail Zymbler (Eds.), Vol. 1618. Springer International Publishing, Cham, 131–142. https://doi.org/10.1007/978-3-031-11623-0_10
- [39] Yousef Saad. 2003. *Iterative Methods for Sparse Linear Systems* (second ed.). Society for Industrial and Applied Mathematics. <https://doi.org/10.1137/1.9780898718003>
- [40] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel Emer. 2020. *Efficient Processing of Deep Neural Networks* (1 ed.). Springer Cham. <https://doi.org/10.1007/978-3-031-01766-7>
- [41] Reginald P. Tewarson. 1973. *Sparse matrices*. Vol. 69. Academic press New York, State University of New York, Stony Brook, NY.
- [42] Richard Vuduc, James W Demmel, and Katherine A Yelick. 2005. OSKI: A library of automatically tuned sparse matrix kernels. *Journal of Physics: Conference Series* 16, 1 (jan 2005), 521. <https://doi.org/10.1088/1742-6596/16/1/071>
- [43] F. Vázquez, E M Garzon, Jose Martinez, and J Fernández. 2009. The sparse matrix vector product on GPUs. *Proceedings of the 2009 International Conference on Computational and Mathematical Methods in Science and Engineering* 2 (01 2009).

- [44] Chen Wang, Chuan Xu, and Abdel Lisser. 2014. Bandwidth Minimization Problem. In *10th International Conference on MOdeling, Optimization and SIMulation - MOSIM14*. Nancy, France. <https://hal.science/hal-01166658>
- [45] Lucas Wilkinson, Kazem Cheshmi, and Maryam Mehri Dehnavi. 2023. Register Tiling for Unstructured Sparsity in Neural Network Inference. *Proc. ACM Program. Lang.* 7, PLDI, Article 188 (June 2023), 26 pages. <https://doi.org/10.1145/3591302>
- [46] Yannan Nellie Wu, Po-An Tsai, Angshuman Parashar, Vivienne Sze, and Joel S. Emer. 2022. Sparseloop: An Analytical Approach To Sparse Tensor Accelerator Modeling. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers and Association for Computing Machinery, New York, NY, USA, 1377–1395. <https://doi.org/10.1109/MICRO56248.2022.00096>
- [47] Wm. A. Wulf and Sally A. McKee. 1995. Hitting the Memory Wall: Implications of the Obvious. *SIGARCH Comput. Archit. News* 23, 1 (mar 1995), 20–24. <https://doi.org/10.1145/216585.216588>
- [48] Ichitaro Yamazaki and Xiaoye S. Li. 2011. On Techniques to Improve Robustness and Scalability of a Parallel Hybrid Linear Solver. In *High Performance Computing for Computational Science – VECPAR 2010*, José M. Laginha M. Palma, Michel Daydé, Osni Marques, and João Correia Lopes (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 421–434.

Contents

Acknowledgements	1
Epigraph	2
Résumé	3
Abstract	4
List of Publications	5
Summary	6
Notations	8
Glossary	9
Acronyms	11
Introduction	12
1 Background	14
1.1 Introduction	14
1.2 Sparse Matrices	15
1.2.1 Banded Matrices	15
1.3 Sparse Formats	16
1.4 Algorithms for Dense and Sparse Matrix Multiplication	17
1.4.0.1 Inner-Product Algorithm	18
1.4.0.2 Outer-Product Algorithm	18
1.4.0.3 Gustavson's Algorithm	18
1.4.0.4 Comparison of the Algorithms	19
1.4.1 Tiling	20
1.5 Hardware Challenges for Sparse Matrix Computation	21
1.5.1 Inner-Product Algorithm	21
1.5.2 Outer-Product Algorithm	21
1.5.3 Summary	22
1.5.4 SpMSPM Algorithm Analysis	24
1.6 Conclusion	27
Conclusion	28
1 TODO	28
Appendices	29
A Sparse Formats	30
COO	30
CSR & CSC	30
BCSR	30
DIA	30
ELL	30

	HYB	30
B	A Generic Representation for <i>Sparse Formats</i>	31
C	Extended Overview of the State-of-the-Art Accelerators for Sparse Computing . .	32
D	Analytical Model of a <i>Reduction Cache</i> Computing an OP SpMV Kernel	33
E	Python Model of a Data-Cache with Support for <i>Reductions</i>	34
F	Statistics	35
List of Figures		36
List of Tables		37
List of Algorithms		38
List of Source Code		39
Bibliography		40
Contents		44

Plan (Brouillon)

- Introduction (~3 p.)
 - "light" > give the entry point on sparse matrices and irregular accesses (1-2 p.)
 - Mains contributions (1 p.)
 - introduce the structure of the manuscript
- Background (~5 p.)
 - irregular accesses -> def, issues (in memory but not only?), domains/applications (including sparse matrix processing) (1 p.)
 - Focus on sparse matrices, context: it is widely used, in many domains, sparse, large, structures
 - Sparse matrices in a kernel: show where the accesses are regular vs irregular (infos on basic sparse formats (other sparse formats in appendix x), tiling, basic kernels, basic algos...) (2-3 p.)
 - Hardware challenges clearly defined (gather, scatter...) (1 p.)
- SoA (~11 p.)
 - Present solutions in SW for CPU/GPU, such as special sparse formats, possibility to play on algo (gust analysis), tiling methods adapted for sparse data (specific sparse format + specific algos) (expliquer clairement que ces solutions sont interessantes et pourquoi on se concentre sur le HW) (1-2 p.)
 - Present solutions in HW (dedicated accelerators) -> comparison table along common characteristics (1-2 p.)
 - Presentation of the major accelerators (selection of 3 important: processor assist PHI, standalone GAMMA, multi algo SPADA, ACES) (others in appendix xi) (3-4 p.)
 - Problem of comparison / comparison of standalone / other analysis (issue of matrix structure) -> Objectives: convince reader that SoA was done in depth, and acknowledge the matrix structure issue (1 p.)
 - Takeaways -> paths for designing an accelerator (end of SoA paper) (1-2 p.)
 - We choose a path -> show which one (coarse grain, at the level of the SoA, comparable to SortCache, PHI...)
- High level architectural presentation of SpDCache (~10 p.)
 - Reminder of the path we choose with more details: cache / reductions / OP SpMV / ... (1 p.)
 - Emphasis the comparison between concerned SoA accelerators -> show what is already done and useful, show what is missing
 - Present the case of banded matrices as an example of structure to leverage (1-2 p.)
 - SpDCache, architecture presentation with two regions, details on REDs transactions, IBRC vs OBRT (7 p.)
 - Initial results from Python model (ASAP) and note that a probabilistic model and RTL model will follow
- Mathematical analysis of the cache memory accesses for processing sparse matrices (~23 p.)
 - Explain the objectives of this chapter (~formal justification of the concept + cache size/region dimensioning) (1 p.)
 - Describe the design space of the path we choose (with reminders if necessary) -> introduce base parameters and/or variables, give the exact algorithm... (1 p.)
 - Show the interest of separate cache regions for A/b and c (formal) -> show that a GC is obviously not efficient when random accesses / confirm why SortCache, PHI, ... exist (2 p.)

- Show that A and b are not a limiting factor with a formal reasoning -> give the needed space for GRC when prefetcher (clear statement that gather is not the aim of the thesis) **(2-3 p.)**
- We use a GRC (no modification) for A and b -> get nb loads
- Show the interest of a reduction cache region for c -> REDs / reduce the number of mem accesses / link to SoA (see ASAP) **(2 p.)**
- We created a probabilistic model of the reduction cache which is based on the following principles ... (presented in detail in appendix n) / advantage of this model / we implemented this model in C / validation of the model with state of the art article on generic cache formalisation / introduce the graphs for the next sections **(1-2 p.)**
- Show that reduction cache does not handle randomness -> model graph **(1-2 p.)**
- Show RedC 1reg vs RedC 2reg on banded example **(2-3 p.)**
- Show best cache dimensions for the dense part (band) **(1 p.)**
- Show that sparse region (out of band) could use an other region for fine grain density (case of sub bands) -> dimensions **(1-2 p.)**
- Show that sparse region could use adapted HW because of the isolated elements -> dimensions **(1-2 p.)**
- Show how it would work with a uniform matrix -> how does it affects the dimensions **(1 p.)**
- Conclude on our solution and dimensioning **(1 p.)**
- Micro architecture of SpDCache **(~10 p.)**
 - Present our hardware solution, based on the concept
 - Start with basic info on HPDCache (only the one that are useful for SpDCache)
 - Describe modif in pipeline / hazard management / throughput
 - Modif in RAM accesses ...
 - Stats: git / bugs / lines of code / synthesis
- RTL simulation results **(~10 p.)**
 - Experimentations: RTL simulation results -> matrices chosen, generated / metrics / ...
 - Results / comparison with theory / analysis / conclusion
- Future enhancement **(~8 p.)**
 - Architecture presentation with three regions (add OBRC, CB...) **(2 p.)**
 - Architecture of eviction anticipation **(2 p.)**
 - Present multicore version -> scale **(2 p.)**
 - Present additional optimisation (region config, learn from first iteration) **(1-2 p.)**
- Conclusions **(~2 p.)**
 - ..
- Appendices
 - All sparse formats **(~5 p.)**
 - Other accelerators of SoA **(~10 p.)**
 - Probabilistic model of the reduction cache **(~10 p.)**
 - Python model **(~1 p.)**